

How Computers Work · Section 2

Binary, Hexadecimal, and Bits

By TOUCH DEN · November 11, 2026

Contents

What You Will Learn 1

Before You Start 1

1. Bits and Bytes 1

2. Counting in Binary 2

3. Converting Between Binary and Decimal 2

4. Hexadecimal, and Why It Is Used 3

5. Units: Kilobytes, Megabytes, Bits vs. Bytes 4

6. Bitwise Thinking: AND, OR, XOR, and NOT 4

7. How Negative Numbers Are Stored 5

8. Reading a Hex Dump 5

Common Mistakes 6

Summary 7

Check Your Understanding 7

Practice 8

What You Will Learn

- What bits and bytes are.
- How to count in binary, and convert between binary and decimal.
- What hexadecimal is, and why security tools use it everywhere.
- Units: kilobytes, megabytes, and the difference between bits and bytes.
- Bitwise thinking: AND, OR, XOR, and NOT.
- How computers store negative numbers.
- How to read a hex dump.

Before You Start

Read lesson 1. You know that inside a computer everything is numbers, stored as on/off switches. This lesson opens up those switches. A little arithmetic is all you need; we build everything from zero. The console examples use Python and the `xxd` tool, and are shown exactly as they ran on a real machine.

---

1. Bits and Bytes

At the very bottom, a computer stores everything using switches that are either **off** or **on**. We write off as **0** and on as **1**. A single one of these switches is called a **bit** (short for "binary digit"). A bit is the smallest piece of information there is: one yes-or-no, one 0-or-1.

One bit alone cannot say much, only two things (0 or 1). So computers group bits together. Eight bits grouped together make one **byte**. A byte is the basic unit a computer works with: files, memory, and network data are all measured in bytes.

How much can one byte hold? Eight bits, each 0 or 1, give 2 × 2 × 2 × 2 × 2 × 2 × 2 × 2 = **256** different patterns, from `00000000` to `11111111`. So one byte can represent 256 different values, usually the numbers 0 to 255. That number, 256, and its top value, 255, will appear again and again.

Term	Meaning
Bit	One switch: 0 or 1
Byte	8 bits grouped together; 256 possible values (0–255)
Nibble	4 bits (half a byte); you will meet this with hex

Everything larger, a letter, a number, a pixel, a whole file, is just many bytes in a row.

2. Counting in Binary

The numbers you use every day are in **decimal** (base 10): ten digits, 0 through 9. When you run out of digits past 9, you add a new column: 10, 11, and so on. Each column is worth ten times the one to its right: ones, tens, hundreds, thousands.

Computers use **binary** (base 2): only two digits, 0 and 1. The idea is identical, but each column is worth **two** times the one to its right, not ten. The columns are 1, 2, 4, 8, 16, 32, 64, 128, each a doubling.

To read a binary number, add up the column values wherever there is a 1:

Binary	128	64	32	16	8	4	2	1	Decimal
`0000 1101`	0	0	0	0	1	1	0	1	8 + 4 + 1 = **13**
`1111 1111`	1	1	1	1	1	1	1	1	**255**
`1000 0000`	1	0	0	0	0	0	0	0	**128**

That is all binary is: the same counting you already know, with two digits instead of ten. Computers use it because a switch has exactly two states, off and on, which map perfectly onto 0 and 1.

3. Converting Between Binary and Decimal

You rarely convert by hand in real work, a tool does it, but doing a couple yourself makes binary click.

Binary to decimal: add the column values where there is a 1. `0000 1101` has 1s in the 8, 4, and 1 columns: 8 + 4 + 1 = 13.

Decimal to binary: find the biggest column value that fits, subtract, and repeat. For 13: the biggest power of two that fits is 8 (leaving 5), then 4 (leaving 1), then 1 (leaving 0). So the 8, 4, and 1 columns are 1: `0000 1101`.

Any tool can check you. Here it is in Python, shown exactly as it ran:

```
$ python3 -c 'print(bin(13), 13); print(int("1010", 2))'
0b1101 13
10
```

The `0b` prefix is how Python (and many tools) say "this is binary". So `0b1101` is binary 1101, which is 13 in decimal, and binary `1010` is 10. You do not need to memorize conversions; you need to understand what the columns mean, and know that every tool can convert for you.

4. Hexadecimal, and Why It Is Used

Binary is correct but hard for humans: `11111111` is eight characters for one byte, and easy to miscount. So security tools almost always show bytes in **hexadecimal** (base 16), usually just called **hex**.

Hex has sixteen digits. The first ten are 0–9, as usual. For the values ten to fifteen, it borrows letters: A = 10, B = 11, C = 12, D = 13, E = 14, F = 15.

Why sixteen? Because 16 is 2×2×2×2, so **one hex digit is exactly four bits** (a nibble), and **two hex digits are exactly one byte.** That neat fit is the whole reason hex is used: it is a short, exact way to write bytes.

Binary (4 bits)	Hex	Decimal
`0000`	0	0
`1010`	A	10
`1111`	F	15
`1111 1111`	FF	255

So one byte, which is `00000000` to `11111111` in binary, is just `00` to `FF` in hex, two characters, always. Hex is often written with a `0x` prefix, so `0xFF` means hex FF, which is 255. Check it in Python:

```
$ python3 -c 'print(hex(255), int("ff", 16))'
0xff 255
```

From now on, whenever you see something like `0x4D` or `4d 69 6e 61`, you are looking at bytes written in hex. Get comfortable with it; it is the everyday language of memory, files, and network data in security.

5. Units: Kilobytes, Megabytes, Bits vs. Bytes

Bytes add up fast, so we use bigger units. Each step is about a thousand times the last (technically 1024, which is 2 to the power of 10, but "about a thousand" is close enough to reason with):

Unit	About how many bytes	Rough example
Kilobyte (KB)	~1,000	A page of text
Megabyte (MB)	~1,000,000	A photo or a song
Gigabyte (GB)	~1,000,000,000	A movie; your phone's memory in GB
Terabyte (TB)	~1,000,000,000,000	A large hard drive

One trap catches everyone: **a bit and a byte are not the same, and the difference shows up in a familiar place.** A byte is 8 bits. By convention, storage (files, disks) is measured in **bytes**, written with a capital B (MB, GB), while network and internet speeds are measured in **bits**, written with a lowercase b (Mbps = megabits per second).

That is why a "100 Mbps" internet connection downloads a file at only about 12 megabytes per second, not 100: you divide by 8, because the speed is in bits and the file is in bytes. Knowing this saves confusion, and it matters when you reason about how long it takes to move data, for example when exfiltrating or capturing it in later courses.

6. Bitwise Thinking: AND, OR, XOR, and NOT

Because everything is bits, computers can work on bits directly, one position at a time. These **bitwise operations** are everywhere in security: in permissions, network masks (lesson on subnetting in the Networking course), simple ciphers, and low-level code.

Line up two bytes and compare each bit position:

- **AND** gives 1 only if **both** bits are 1. (Used to *check* or *clear* bits.)
- **OR** gives 1 if **either** bit is 1. (Used to *set* bits.)
- **XOR** (exclusive or) gives 1 if the bits are **different**. (Used in simple encryption and to flip bits.)
- **NOT** flips every bit: 0 becomes 1, 1 becomes 0.

Here they are on two 4-bit values, run for real:

```
$ python3 -c 'a,b=0b1100,0b1010
print(f"{a:04b} AND {b:04b} = {a&b:04b}")
print(f"{a:04b} OR  {b:04b} = {a|b:04b}")
print(f"{a:04b} XOR {b:04b} = {a^b:04b}")'
1100 AND 1010 = 1000
1100 OR  1010 = 1110
1100 XOR 1010 = 0110
```

Read the AND line: only the leftmost position has a 1 in *both* numbers, so only that position is 1 in the answer. **XOR** deserves special attention: it is the heart of many simple ciphers, because applying the same XOR twice returns the original value. You will meet it again in the Cryptography course. For now, just recognize what AND, OR, and XOR do to bits.

7. How Negative Numbers Are Stored

So far our bytes held 0 to 255, all positive. But programs need negative numbers too. Computers store them with a clever trick called **two's complement**. You do not need the full method yet, but you should know the idea and one consequence.

The idea: for a signed byte, the top bit is used as a sign. If it is 0, the number is positive (0 to 127). If it is 1, the number is negative (−1 to −128). The exact pattern is chosen so that ordinary addition still works without special cases.

The consequence worth remembering: **the same byte can mean different numbers depending on whether it is read as signed or unsigned.** The byte `1111 1111` is 255 if read as an unsigned number, but −1 if read as a signed number. The bits are identical; only the *interpretation* differs. This is exactly the lesson-1 idea, numbers mean nothing until something decides how to read them, and it is the source of a whole family of bugs (integer overflows and sign errors) that you will study in Binary Exploitation. For now: a byte's bits are fixed, but what number they *represent* depends on how the program reads them.

8. Reading a Hex Dump

Putting it together: a **hex dump** is a view of raw bytes, shown in hex, that security people use constantly, to inspect files, memory, and network captures. Almost every tool can produce one. Here is the text `Hi!`, dumped with the `xxd` tool:

```
$ printf 'Hi!' | xxd
00000000: 4869 21                                  Hi!
```

Read it in three columns:

- `00000000` on the left is the **offset**: how far into the data we are, in hex. Here, the start (byte 0).
- `4869 21` in the middle is the **bytes themselves, in hex**. `48`, `69`, `21`, three bytes for three characters.
- `Hi!` on the right is those same bytes shown as text, where they are printable.

So the letter `H` is the byte `0x48`, `i` is `0x69`, and `!` is `0x21`. (Why those particular numbers? That is text encoding, the subject of the next lesson.) Here is a four-byte example:

```
$ printf 'Mina' | xxd
00000000: 4d69 6e61                                Mina
```

`M` = `0x4d`, `i` = `0x69`, `n` = `0x6e`, `a` = `0x61`. Being able to read a dump like this, offset, hex bytes, and text, is a core skill. A reverse engineer reads program bytes this way, a forensic analyst reads disk and memory this way, and a network analyst reads captured packets this way. You now have the alphabet for all of it. The next lesson explains *why* `H` is `0x48`: how text becomes numbers.

Common Mistakes

1. Confusing bits and bytes. A byte is 8 bits. Storage is in bytes (MB), network speed is in bits (Mbps). Divide network speed by 8 to compare.

2. Thinking hex is a different kind of number. Hex is just a short, exact way to *write* the same bytes. One byte is always two hex digits.

3. Trying to memorize conversions. Understand what the columns mean; let a tool do the arithmetic. Nobody converts long binary by hand in real work.

4. Forgetting that a byte's meaning depends on interpretation. `1111 1111` is 255 unsigned or −1 signed. The bits are the same; the reading differs.

5. Miscounting offsets in a hex dump. The offset on the left is in hex, and counts from 0. The byte at offset `0x10` is the 17th byte, not the 10th.

Summary

Term	Remember
Bit / byte	One switch (0/1) / 8 bits, 256 values (0–255)
Binary	Base 2; columns double: 1, 2, 4, 8, 16, …
Hex	Base 16; one digit = 4 bits, two digits = one byte; 0x prefix
Units	KB/MB/GB in bytes (capital B); network speed in bits (lowercase b)
Bitwise	AND (both), OR (either), XOR (different), NOT (flip)
Signed vs unsigned	Same bits can mean different numbers (e.g. 255 vs −1)
Hex dump	Offset · bytes in hex · text; the everyday view of raw data

Check Your Understanding

1. How many bits are in a byte, and how many different values can one byte hold?
2. Convert binary `0010 0110` to decimal. (Hint: use the column values.)
3. Why do security tools show bytes in hex instead of binary?
4. How many bits is one hex digit? How many hex digits is one byte?
5. Your internet is "80 Mbps". Roughly how many megabytes per second is that, and why?
6. What does XOR do to two bits, and why is it useful in simple ciphers?
7. In the hex dump `00000000: 4d69 6e61 Mina`, what byte (in hex) is the letter `n`?

Answers

1. 8 bits; 256 different values (0 to 255).
2. The 1s are in the 32, 4, and 2 columns: 32 + 4 + 2 = 38.
3. Because binary is long and easy to miscount (8 characters per byte), while hex is short and exact: two hex digits per byte, with one digit per 4 bits.
4. One hex digit is 4 bits (a nibble); one byte is two hex digits.
5. About 10 MB/s. Network speed is measured in bits and files in bytes, so you divide by 8: 80 ÷ 8 = 10.
6. XOR gives 1 when the two bits differ. It's useful because applying the same XOR value twice returns the original, so the same key both scrambles and unscrambles.
7. `0x6e` (shown as `6e` in the dump). `n` is the third byte: `4d 69 6e 61` → M, i, n, a.

Practice

1. By hand, convert the decimal number **42** to an 8-bit binary number, using the column values (128, 64, 32, 16, 8, 4, 2, 1). Then convert it to two hex digits.
2. Line up `0b1011` and `0b0110`. Work out, by hand, the result of AND, OR, and XOR on them.
3. If you have a terminal, run `printf 'Den' | xxd` and read the dump: what are the three bytes, in hex? (If you cannot run it, reason about what the columns would show.)

A Worked Example

1. 42: the biggest column that fits is 32 (leaving 10), then 8 (leaving 2), then 2 (leaving 0). So the 32, 8, and 2 columns are 1: `0010 1010`. In hex, split into two nibbles: `0010` = 2 and `1010` = A, so `0x2A`.
2. `1011` AND `0110` = `0010` (only the 2-column is 1 in both). `1011` OR `0110` = `1111`. `1011` XOR `0110` = `1101` (1 where they differ).
3. `printf 'Den' | xxd` gives `44 65 6e`: `D` = `0x44`, `e` = `0x65`, `n` = `0x6e`.

Course mark icon from Material Symbols by Google, Apache License 2.0.